

Comparing algorithms for computing homography between 2 images (8h)

Objective

The objective of the lab work is to compute the homography between two images `img1` and `img2`. Given a set of points $\mathbf{x}_i$ in `img1` (of the space $\mathbb{P}^2$) and a corresponding set of points $\mathbf{x}'_i$ likewise in `img2` (of the space $\mathbb{P}^2$) the problem consists in computing the projective transformation, described by the matrix $\mathbf{H}$, that maps each $\mathbf{x}_i$ to $\mathbf{x}'_i$:

$$\mathbf{x}'_i = \mathbf{H}\mathbf{x}_i$$

The space $\mathbb{P}^2$ means the image points are expressed in homogeneous coordinates, i.e

$$\mathbf{x}_i = (x_i, y_i, 1)^\top$$

$$\mathbf{x}'_i = (x'_i, y'_i, 1)^\top$$

then $\mathbf{H}$ is such that

$$\mathbf{H} = \begin{pmatrix} h_1 & h_2 & h_3 \\ h_4 & h_5 & h_6 \\ h_7 & h_8 & h_9 = 1 \end{pmatrix}$$

The matrix $\mathbf{H}$ can also be represented by a 9-vector such that

$$\mathbf{H} = \begin{pmatrix} \mathbf{h}^1 \\ \mathbf{h}^2 \\ \mathbf{h}^3 \end{pmatrix} = \begin{pmatrix} h_1 \\ h_2 \\ \vdots \\ h_8 \\ h_9 = 1 \end{pmatrix}$$

Homography estimation solutions

Homography estimation solutions implemented in OpenCV

Analyse the code proposed in OpenCV documentation:

https://docs.opencv.org/4.x/d1/de0/tutorial_py_feature_homography.html

and more particularly the function `findHomography()` which implements the following solutions: ordinary least squares method, RANSAC and LMEDS.

Homography estimation using DLT

According to (Hartley and Zisserman (2006)) (pages 87-110) for a single point correspondence one can say that the cross vector product is such that

$$\mathbf{x}'_i \times \mathbf{H}\mathbf{x}_i = 0$$

After simplification this equation becomes

$$\mathbf{A}_i \mathbf{h} = 0$$

with $\mathbf{A}_i$ a 2×9 matrix such that

$$\mathbf{A}_i = \begin{pmatrix} 0 & 0 & 0 & \mathbf{x}_i^\top & 1 & y'_i \mathbf{x}_i^\top & y'_i \\ \mathbf{x}_i^\top & 1 & 0 & 0 & 0 & x'_i \mathbf{x}_i^\top & x'_i \end{pmatrix}$$

The concatenation of the set of $\mathbf{A}_i$ leads to the *Direct Linear Transform (DLT)*

$$\mathbf{A} \mathbf{h} = 0$$

The basic DLT algorithm for computing $\mathbf{H}$ is given by (Algorithm 1).

Data: $n \geq 4$ 2D to 2D point correspondences $\mathbf{x}_i \leftrightarrow \mathbf{x}'_i$

Result: The 2D homography matrix $\mathbf{H}$ such that $\mathbf{x}'_i = \mathbf{H}\mathbf{x}_i$

1. For each correspondence $\mathbf{x}_i \leftrightarrow \mathbf{x}'_i$ compute the matrix $\mathbf{A}_i$;
2. Assemble the $n(2 \times 9)$ matrices $\mathbf{A}_i$ into a single $2n \times 9$ matrix $\mathbf{A}$;
3. Find the SVD $\mathbf{U}\mathbf{D}\mathbf{V}^\top$ of $\mathbf{A}$; $\mathbf{h}$ is the last column of $\mathbf{V}$;
4. The matrix $\mathbf{H}$ is determined from $\mathbf{h}$ as described above ;

Algorithm 1: The basic DLT algorithm for computing $\mathbf{H}$

However, it is desirable to make the homography invariant to translation and scale of the point coordinates: this can be done by *normalizing* the points. This leads to the (Algorithm 2).

Data: $n \geq 4$ 2D to 2D point correspondences $\mathbf{x}_i \leftrightarrow \mathbf{x}'_i$

Result: The 2D homography matrix $\mathbf{H}$ such that $\mathbf{x}'_i = \mathbf{H}\mathbf{x}_i$

1. Normalization of the set of $\mathbf{x}_i$: Compute a similarity transformation $\mathbf{T}$, consisting of a translation and scaling, that takes points $\mathbf{x}_i$ to a new set of points $\tilde{\mathbf{x}}_i$ such that the centroid of the points $\tilde{\mathbf{x}}_i$ is the coordinate origin $(0, 0)^\top$, and their average distance from the origin is $\sqrt{2}$;
2. Normalization of the set of $\mathbf{x}'_i$: Compute a similar transformation $\mathbf{T}'$ for the points in the second image, transforming points $\mathbf{x}'_i$ to $\tilde{\mathbf{x}}'_i$;
3. DLT: Apply the basic algorithm to the correspondences $\tilde{\mathbf{x}}_i \leftrightarrow \tilde{\mathbf{x}}'_i$ to obtain a homography $\tilde{\mathbf{H}}$;
4. Denormalization: Set $\mathbf{H} = \mathbf{T}'^{-1} \tilde{\mathbf{H}} \mathbf{T}$;

Algorithm 2: The normalized DLT algorithm for computing $\mathbf{H}$

Application

Working image acquisition

Following the example given in

https://docs.opencv.org/4.x/d1/de0/tutorial_py_feature_homography.html

you can acquire with a phone the image of a **planar surface** (e.g book, etc.) alone (`img1` or query image) and in a scene (`img2` or train image), and transfer the both images into the computer for processing.

Alternatively, you can download the images available on the moodle of the course. Homography estimation solutions implemented in OpenCV.

Homography estimation using RANSAC

Run the code with obtained images using JupyterLab with OpenCV (or Spyder with OpenCV)

Modify the code to display the value of the homography matrix and its trace

Plot the trace of the homography for 15 executions

Homography estimation using LMEDS

Replace RANSAC by LMEDS and plot the trace of the homography for 15 executions.

Homography estimation using ordinary least squares

Replace LMEDS by Least squares and plot the trace of the homography for 15 executions.

Homography estimation using DLT

Code the basic DLT algorithm

Code the normalized DLT algorithm, if time

Conclusion

Conclude on the performance of the methods, more particularly their repeatability measured with the dispersion of the matrix trace.